

Proyecto Final Fundamentos de los Sistemas Operativos

Aplicación Blockchain

Guzman Pérez

Santiago Urtiaga

Introducción:

La intención de este proyecto es crear una aplicación Blockchain funcional, en la que se puedan hacer transacciones, agruparlas en bloques y añadirlos a la cadena de forma segura. En esta aplicación se permite la conexión de diferentes nodos, que pueden minar bloques, calculando un hash valido y construir una Blockchain compartida por todos ellos.

Metodología:

Se ha implementado un primer módulo en el que se han creado las clases Bloque y Blockchain.

Cada bloque queda definido por:

índice: un entero que identifica el bloque,

transacciones: una lista con las transacciones del bloque. (cada transacción se ha tratado como un diccionario),

timestamp: momento del tiempo en el que se ha generado el bloque,

hash_previo: hash del bloque anterior para asegurar que la Blockchain no está corrupta,

prueba: entero que se utiliza para garantizar que el hash empiece por un determinado número de ceros (es un parametro de seguridad).

La clase bloque cuenta con una única función que calcula el hash del bloque.

La Blockchain queda definida por:

Dificultad: número de ceros que debe contener el hash al inicio como medida de seguridad,

Transaction_list, inicialmente vacía, lista con las transacciones que se incluirán en el nuevo bloque.

Block_list, lista de bloques enlazados por el hash (el hash de cada bloque debe coincidir con el hash_anterior del siguiente,

Las funciones de Blockchain son:

Primer bloque que genera un primer bloque con el que empezar la blockchain,

Nueva transacción, que añade una transacción a la lista de transacciones,

Nuevo bloque, que permite crear un bloque que posteriormente será añadido,

Prueba de trabajo, que se asegura de que el hash de cada bloque comienza por dificultad número de ceros,

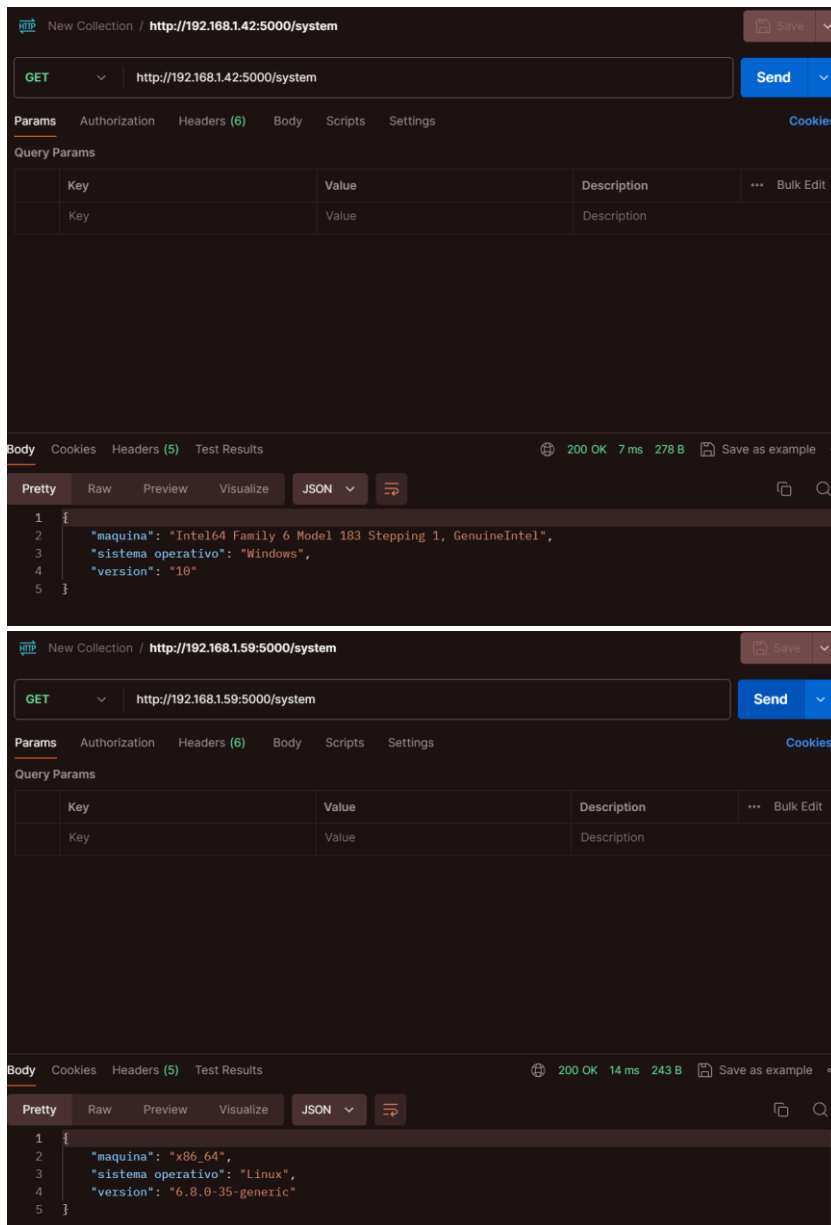
Prueba válida, que verifica que se cumpla la condición de prueba de trabajo y que el hash anterior sea correcto,

Integrar bloque, que añade el bloque a la lista de bloques.

Una vez se tienen las clases bloque y blockchain, se crea una API donde se puedan hacer distintas peticiones a dicha blockchain.

Las distintas peticiones que se pueden hacer son:

/system, que permite conocer el sistema operativo, procesador y version del SO de la máquina:



/transacciones/nueva, que permite crear una transacción dado un origen, un destino y una cantidad:

HTTP New Collection / **http://192.168.1.42:5000/transacciones/nueva** Save

POST ▼ **http://192.168.1.42:5000/transacciones/nueva** Send ▼

Params Authorization Headers (8) **Body** ● Scripts Settings Cookies

Query Params

	Key	Value	Description ⓘ	...	Bulk Edit
	Key	Value	Description		

Body Cookies Headers (5) Test Results 200 OK 8 ms 233 B Save as example

Pretty Raw Preview Visualize **JSON** ▼ ⌵

```
1 {  
2   "mensaje": "La transaccion se incluirea en el bloque con indice 1"  
3 }
```

HTTP New Collection / **New Request** Save

POST ▼ **http://192.168.1.59:5000/transacciones/nueva** Send ▼

Params Authorization Headers (8) **Body** ● Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Beautify

```
1 {  
2   ...."origen": "192.168.1.42",  
3   ...."destino": "192.168.1.58",  
4   ...."cantidad": 100  
5 }
```

Body Cookies Headers (5) Test Results 200 OK 14 ms 232 B Save as example

Pretty Raw Preview Visualize **JSON** ▼ ⌵

```
1 {  
2   "mensaje": "La transaccion se incluirea en el bloque con indice 1"  
3 }
```

/chain, un método get que obtiene la cadena completa del nodo en cuestión:

GET http://192.168.159:5000/chain Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
Key	Value	Description		

body Cookies Headers (5) Test Results 200 OK 14 ms 507 B Save as example

Pretty Raw Preview Visualize JSON

```
2  "chain": [  
3    {  
4      "hash_previo": 1,  
5      "indice": 1,  
6      "prueba": 0,  
7      "timestamp": 1717616536.1861522,  
8      "transacciones": null  
9    },  
10   {  
11     "hash_previo": "36b3b2dc35d8ceb81968d3039cc6c522592aef7e2829ffb78b85adf544bbb844",  
12     "indice": 2,  
13     "prueba": 64816,  
14     "timestamp": 1717616602.4600132,  
15     "transacciones": [  
16       {  
17         "cantidad": 100,  
18         "destino": "192.168.1.58",  
19         "origen": "192.168.1.42"  
20       }  
21     ]  
22   },  
23 ],  
24 "longitud": 2
```

/minar, que se asegura de que haya transacciones para minar, que la blockchain sea la más larga (en caso de no serlo se actualiza a la más larga) y añade un bloque a la Blockchain, obteniendo una compensación por ello

New Collection / http://192.168.159:5000/minar Save

GET http://192.168.159:5000/minar Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
Key	Value	Description		

Body Cookies Headers (5) Test Results 200 OK 329 ms 278 B Save as example

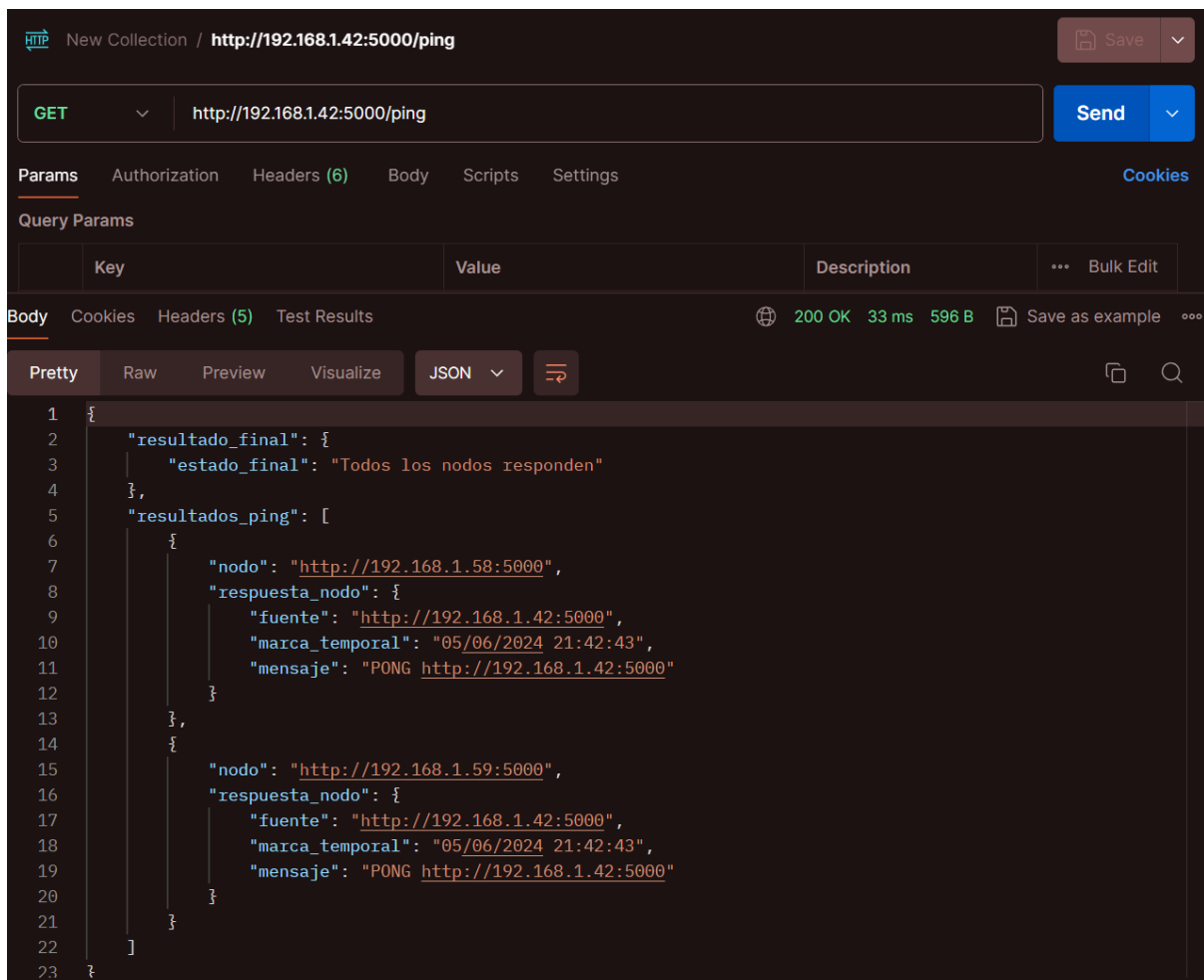
Pretty Raw Preview Visualize JSON

```
1 {  
2   "mensaje": "ha recibido una compensación por minar el bloque, esta se incluíra en el bloque con índice  
3   3"  
3 }
```

También se ha creado un hilo que obtenga la blockchain y la guarde en un archivo de seguridad que se sobrescribe cada minuto.

Por último, y más importante de todo, se han generado funciones para poder pasar de una api centralizada a una descentralizada; es decir, ahora se puede correr desde distintos dispositivos y/o máquinas virtuales, pudiendo correrse en el host, guest ambos...

Para ello hemos diseñado una función /nodos/regar, a la que se le añade la ip y los puertos de los distintos nodos donde se va a ejecutar la api, y que envía una copia de la Blockchain del host (nodo principal, no necesariamente el host de las máquinas virtuales) y tras recibir la respuesta notifica sobre el estado de la conexión.

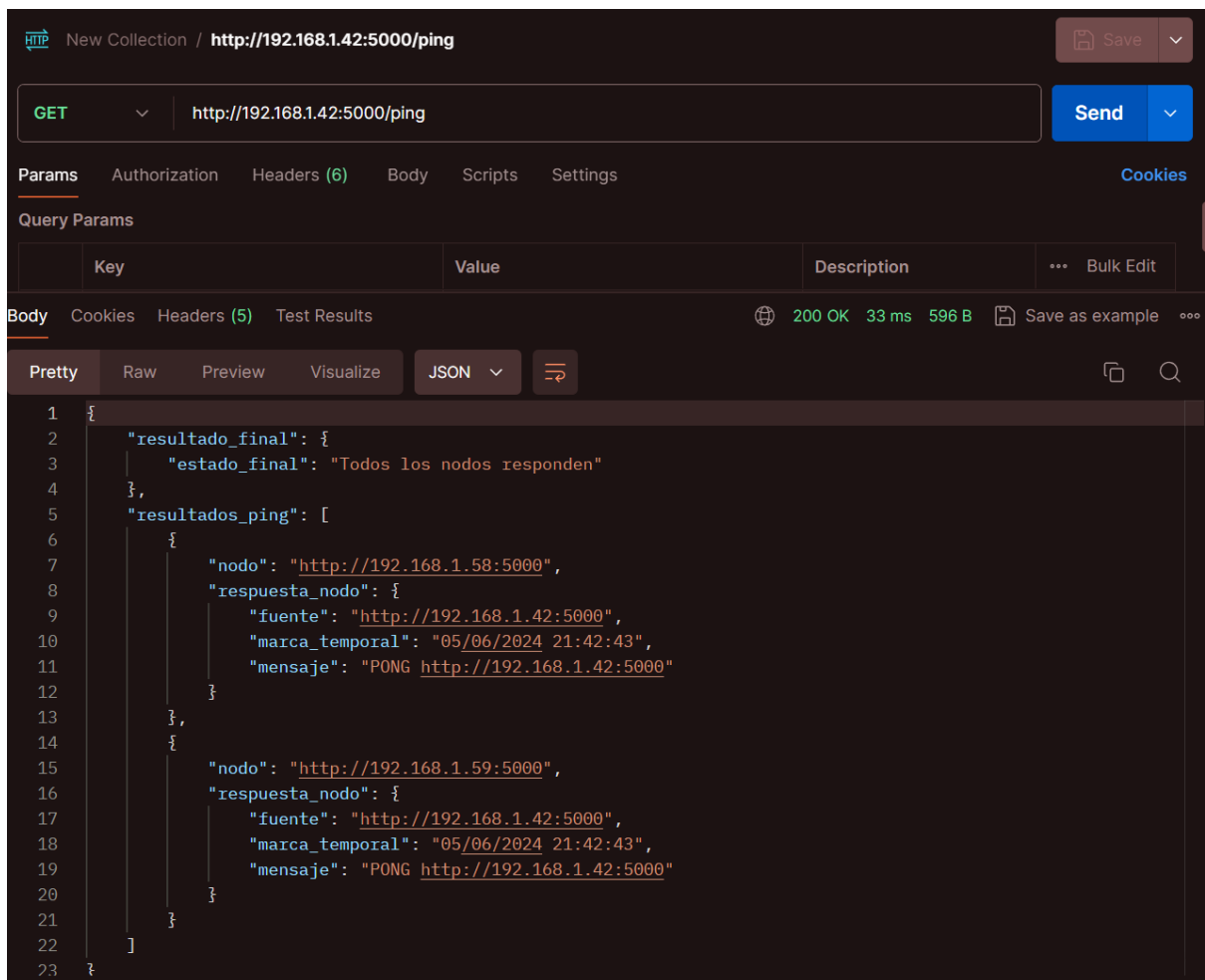


The screenshot shows a REST client interface with the following details:

- URL:** `http://192.168.1.42:5000/ping`
- Method:** GET
- Status:** 200 OK, 33 ms, 596 B
- Response Body (JSON):**

```
1 {
2   "resultado_final": {
3     "estado_final": "Todos los nodos responden"
4   },
5   "resultados_ping": [
6     {
7       "nodo": "http://192.168.1.58:5000",
8       "respuesta_nodo": {
9         "fuente": "http://192.168.1.42:5000",
10        "marca_temporal": "05/06/2024 21:42:43",
11        "mensaje": "PONG http://192.168.1.42:5000"
12      }
13    },
14    {
15      "nodo": "http://192.168.1.59:5000",
16      "respuesta_nodo": {
17        "fuente": "http://192.168.1.42:5000",
18        "marca_temporal": "05/06/2024 21:42:43",
19        "mensaje": "PONG http://192.168.1.42:5000"
20      }
21    }
22  ]
23 }
```

También se ha generado un protocolo para asegurar la conectividad entre los distintos nodos mediante un Ping, que envía unos datos y espera la respuesta (pong) de los distintos nodos con los datos para asegurar que la información llega adecuadamente.



Se adjunta por último una imagen de dos máquinas virtuales ubuntu corriendo el código conjuntamente con la máquina host (windows) y la aplicación postman con la que se han realizado las peticiones

